DoofusAI SPM

AI Security Posture Management

User Manual & Configuration Guide

Author: Shahryar Jahangir
Version 1.1.3 · March 2026

CONFIDENTIAL — FOR INTERNAL USE

Table of Contents

Table of Contents 3

1. Introduction 11

DoofusAI SPM is an open-source AI Security Posture Management platform designed to help organisations identify, prioritise, and remediate security risks across their AI and machine-learning infrastructure. 11

1.1 About This Product 11

As organisations deploy large language models (LLMs), AI agents, machine-learning pipelines, and AI-integrated applications, they face an entirely new category of security risk that traditional cloud security tools were not designed to address. Prompt injection, model supply-chain attacks, insecure retrieval pipelines, and overly privileged AI agents are now real threats — yet most security teams lack specialised tooling to detect them. 11

DoofusAI SPM was created by Shahryar Jahangir to bridge that gap. It provides a unified, configurable scanning platform that assesses AI/ML infrastructure across four target categories, maps every finding to internationally recognised security frameworks, scores risk using a CVSS-style model, and presents results in a clear, actionable dashboard. The tool is designed to be approachable for practitioners who are not security specialists, while remaining precise enough to satisfy rigorous security requirements.11

Key characteristics of DoofusAI SPM: 11

1.2 Author 11

DoofusAI SPM was designed, architected, and built by Shahryar Jahangir. The platform reflects Shahryar's background in cloud security, AI infrastructure, and developer tooling, and is intended as a production-grade foundation that security and platform engineering teams can build upon. 11

1.3 What DoofusAI SPM Scans 11

The platform supports four target types, covering the full AI/ML delivery chain: 11

1.4 Security Controls Implemented 12

DoofusAI SPM implements 12 built-in security checks across the four target types, plus four open-source tool wrappers. Each check implements one or more of the following control categories: 12

1.5 Standards and Sources 13

Every finding produced by DoofusAI SPM is mapped to one or more of the following authoritative frameworks: 13

Open-source security tools integrated as check modules: 13

2. Getting Started 14

This section walks through installing DoofusAI SPM, starting the application, and running your first scan. No prior security experience is required. 14

2.1 System Requirements 14

2.2 Installation 14

1. Download and unzip the package: 14

2. Install all dependencies. Run these commands in order from inside the project folder: 14

ⓘ No C++ compiler or native build tools are required. DoofusAI SPM uses sql.js (pure WebAssembly SQLite) to avoid native compilation errors on newer versions of Node. 14

3. Create your environment configuration file: 15

Open the .env file in any text editor and fill in the relevant values. See Section 3 (Configuration) for a full description of every option. 15

4. Start the application: 15

You should see output like this within a few seconds: 15

5. Open http://localhost:5173 in your browser. The DoofusAI SPM dashboard will load. 15

⚠ The DEP0169 deprecation warning about url.parse() is harmless. It comes from the swagger-ui-express library and does not affect functionality. 15

2.3 Installing Optional Open-Source Tools 15

The four open-source tool integrations are optional. The application runs without them; checks simply report that the tool is not installed. To enable them: 15

ⓘ Each tool installs independently. You can install only the ones you need — for example, just detect-secrets for credential scanning without the full garak probing suite. 16

3. Configuration 17

DoofusAI SPM is configured through environment variables and YAML policy files. This section explains every option. 17

3.1 Environment Variables (.env) 17

The .env file in the project root controls how the application runs. Never commit this file to version control — it may contain API keys. 17

Server Settings 17

API Keys 17

API keys are never stored in the database. They are read from environment variables at runtime and injected into scans by name. Set the keys for whichever providers you plan to scan: 17

⚠ Never paste API keys directly into target configuration forms in the UI.

Always set them here in the .env file and reference them by the variable name in your scan target config. 17

3.2 Policy Files 18

Policies are YAML files that define what DoofusAI SPM checks and how it scores findings. Built-in policies live in engine/policy/. Your custom policies go in engine/custom-policies/ — this folder is excluded from version control by .gitignore. 18

Policy File Format 18

Every policy file must contain the following fields: 18

Valid Values 18

Hot-Reloading Policies 19

You can add or edit policy files without restarting the server. After saving a new .yaml file to engine/custom-policies/, call the reload endpoint: 19

ⓘ Custom policies with the same id as a built-in policy will override the

built-in. This lets you customise remediation text or adjust severity without modifying the engine/policy/ directory. 19

3.3 Writing a Custom Check Module 19

A check module is a JavaScript file placed in engine/checks/. It must export a run function that accepts a target and returns an array of findings: 19

Each check runs in an isolated worker thread with a 30-second timeout. A check that crashes or hangs cannot affect other checks or the server. 20

4. Using the Dashboard 21

The DoofusAI SPM dashboard is a web application that runs at http://localhost:5173. It provides five main screens accessible from the top navigation bar. 21

4.1 Dashboard (Home Screen) 21

The home screen gives you an at-a-glance view of your overall AI security posture. It contains: 21

Posture Score Gauge 21

The circular gauge in the top-left shows your current posture score from 0 to 100. The score is calculated from all open findings using a weighted formula — each finding's severity and confidence level contribute to the penalty. The colour and tier label change based on the score: 21

Summary Cards 21

Four metric cards appear at the top of the screen: Open Findings (total findings not yet resolved), Critical (open findings at critical severity), Scans (7d) (number of scans run in the past week), and Policies Active (number of enabled policies). 21

Charts 21

Top Critical Findings Table 21

The table at the bottom of the dashboard shows the five most severe open findings. Click any row to open the Finding Detail Drawer (see Section 4.3).21

4.2 Scan Configuration Screen 22

Use this screen to configure and launch new scans. A three-step wizard guides you through the process. 22

Step 1 — Select Target Type 22

Choose the type of AI resource you want to scan from the four tiles: LLM / GenAI API, AI Infrastructure, Model Registry, or AI-Integrated App. 22

Step 2 — Configure Target 22

Fill in the target details. The fields shown depend on the target type selected: 22

⚠ API keys must never be entered in the configuration form. Set them as environment variables in .env and the engine will read them automatically.22

Step 3 — Select Policies 22

Choose which security checks to run. Only policies applicable to the selected target type are shown. All enabled policies are selected by default. You can deselect individual policies to skip specific checks. 22

Click Run Scan Now to start the scan. The scan runs in the background; the Recent Scans table at the top of the screen updates automatically when it completes. 22

4.3 Findings Screen 22

The Findings screen lists all security findings across all scans. You can filter, sort, and act on findings from this screen. 22

Filtering Findings 23

Use the filter bar at the top of the screen to narrow the list: 23

Sorting 23

Click any column header to sort by that column. Click again to reverse the sort order. A small arrow indicates the active sort column and direction. 23

Finding Detail Drawer 23

Click any row to open the detail drawer on the right side of the screen. The drawer shows: 23

Acting on Findings 23

Three action buttons appear at the bottom of the detail drawer: 23

4.4 Policy Editor Screen 23

The Policy Editor provides a built-in YAML editor for creating and validating custom policies directly in the browser. You do not need to edit files manually. 23

4.5 Reports Screen 24

The Reports screen generates a formatted security report from any completed scan. 24

Generating a Report 24

Exporting 24

5. Built-In Security Checks 25

DoofusAI SPM includes 12 built-in checks and 4 open-source tool wrappers. This section describes each one. 25

5.1 Severity and Scoring 25

Each finding is assigned a severity level and a confidence level. Together they produce a CVSS-style composite score from 0 to 10: 26

Confidence levels modify the severity score: Confirmed multiplies by 1.0, Probable by 0.75, and Possible by 0.5. For example, a high-severity finding with probable confidence scores 8.0 × 0.75 = 6.0. 26

The overall posture score is calculated as: 26

ⓘ Resolved findings are excluded from the posture score. Acknowledged findings are included — acknowledging a finding records awareness but does not credit the remediation. 26

6. REST API Reference 27

All dashboard functionality is available via REST API. The interactive API documentation (Swagger UI) is available at http://localhost:3001/api/docs while the server is running. 27

6.1 Endpoints 27

6.2 Starting a Scan via API 27

Send a POST request to /api/v1/scans with a target object: 27

The response includes a scan_id. Poll GET /api/v1/scans/:id until the status field changes from running to completed. 28

6.3 Target Configuration Fields 28

LLM API Target 28

AI Infrastructure Target 28

Model Registry Target 28

AI App Target 29

7. Troubleshooting 30

7.1 Common Errors and Fixes 30

7.2 Checking Server Logs 30

All server activity is logged to the terminal where you ran npm run dev. The [0] prefix indicates API server output; [1] indicates the Vite UI server. Look for lines prefixed with [DB], [API], [Scanner], [Runner], or [PolicyLoader] for diagnostic information. 30

To start only the API server (for debugging without the UI): 30

7.3 Resetting the Database 30

The SQLite database is stored at data/doofusai.db. To reset all scan history and findings, stop the server and delete this file: 30

⚠ Deleting the database permanently removes all scan history and finding statuses. This cannot be undone. 31

8. Security and Privacy 32

8.1 How Secrets Are Protected 32

DoofusAI SPM is designed so that sensitive credentials cannot leak through the tool itself: 32

8.2 Data That Stays Local 32

DoofusAI SPM does not make any outbound network calls except to the targets you explicitly configure for scanning. No telemetry, no analytics, no licence checks. All scan data, findings, and reports remain in the local SQLite database on your machine. 32

8.3 Running in Production 32

For production deployments (e.g. running DoofusAI SPM as a shared team service), consider the following additional steps: 32

9. Glossary 33

10. About 35

10.1 Author 35

DoofusAI SPM was designed, architected, and built by Shahryar Jahangir. The product reflects Shahryar's work across cloud security, AI infrastructure, and developer tooling, and is released as a production-grade open platform for security and engineering teams to build upon. 35

10.2 Version History 35

10.3 Licence 35

DoofusAI SPM is released for internal use. The open-source tools integrated as check modules (garak, llm-guard, detect-secrets, checkov) are governed by their respective licences. 35

This document and all associated source code are the intellectual property of Shahryar Jahangir. 35

DoofusAI SPM · User Manual & Configuration Guide · Shahryar Jahangir · March 2026 35

1. Introduction

DoofusAI SPM is an open-source AI Security Posture Management platform designed to help organisations identify, prioritise, and remediate security risks across their AI and machine-learning infrastructure.

1.1 About This Product

As organisations deploy large language models (LLMs), AI agents, machine-learning pipelines, and AI-integrated applications, they face an entirely new category of security risk that traditional cloud security tools were not designed to address. Prompt injection, model supply-chain attacks, insecure retrieval pipelines, and overly privileged AI agents are now real threats — yet most security teams lack specialised tooling to detect them.

DoofusAI SPM was created by Shahryar Jahangir to bridge that gap. It provides a unified, configurable scanning platform that assesses AI/ML infrastructure across four target categories, maps every finding to internationally recognised security frameworks, scores risk using a CVSS-style model, and presents results in a clear, actionable dashboard. The tool is designed to be approachable for practitioners who are not security specialists, while remaining precise enough to satisfy rigorous security requirements.

Key characteristics of DoofusAI SPM:

- Works against live AI infrastructure — not just static code
- Produces findings with step-by-step remediation guidance
- Maps every issue to OWASP, MITRE ATLAS, and NIST AI RMF
- Extensible via YAML policy files — no code changes required
- Integrates leading open-source security tools as pluggable check modules
- Runs entirely on-premises — no data leaves your environment

1.2 Author

DoofusAI SPM was designed, architected, and built by Shahryar Jahangir. The platform reflects Shahryar's background in cloud security, AI infrastructure, and developer tooling, and is intended as a production-grade foundation that security and platform engineering teams can build upon.

1.3 What DoofusAI SPM Scans

The platform supports four target types, covering the full AI/ML delivery chain:

Target Type	Examples	What Is Assessed
LLM / GenAI APIs	OpenAI, Anthropic, Azure OpenAI, Bedrock, Vertex AI	System prompt hardening, API key exposure, rate-limit guardrails, output filtering
AI Infrastructure	MLflow, Jupyter, SageMaker, Kubeflow, GPU clusters	Authentication on ML endpoints, storage encryption, network isolation, audit logging
Model Registries	HuggingFace Hub, MLflow Registry, SageMaker Registry	Model signing, pickle-format risk, SBOM presence, licence declarations
AI-Integrated Apps	RAG pipelines, LLM agents, chatbots, copilots	Prompt injection vectors, PII leakage, excessive agent permissions, insecure retrieval

1.4 Security Controls Implemented

DoofusAI SPM implements 12 built-in security checks across the four target types, plus four open-source tool wrappers. Each check implements one or more of the following control categories:

Control Category	Description
Access Control	Verifies that ML endpoints require authentication and that model artefact storage is not publicly accessible
Data Protection	Checks that training data, model weights, and inference outputs are encrypted at rest and in transit
Input Validation	Detects the absence of prompt injection defences, input scanning, and content filtering on LLM inputs
Output Filtering	Checks for PII scanning, toxicity filtering, and output validation before responses reach end users
Supply Chain Integrity	Verifies model signatures, checksums, SBOM presence, and detects pickle-format artefacts
Secrets Management	Detects API keys and credentials in source code, configuration files, and git history
Least Privilege	Checks that AI agents have only the tool access strictly required for their task
Audit & Logging	Confirms that audit logging is enabled on ML infrastructure and model registries
Human Oversight	Verifies that high-risk agent actions require human approval before execution
IaC Compliance	Scans infrastructure-as-code files for AI/ML-specific misconfigurations

1.5 Standards and Sources

Every finding produced by DoofusAI SPM is mapped to one or more of the following authoritative frameworks:

Framework	Publisher	Role in DoofusAI SPM
OWASP LLM Top 10 (2025)	OWASP Foundation	Primary check catalogue — LLM01 through LLM10 map directly to built-in checks
MITRE ATLAS	MITRE Corporation	Adversarial ML tactic and technique IDs (e.g. AML.T0051) provide attacker-perspective context
NIST AI RMF 1.0	NIST	Risk management functions (GOVERN, MAP, MEASURE, MANAGE) frame findings in governance terms
CIS Benchmarks	Center for Internet Security	Referenced indirectly via checkov IaC checks for AWS, Azure, and GCP AI services
Custom Policies	User-defined	YAML/JSON files that extend or override any built-in check with organisation-specific rules

Open-source security tools integrated as check modules:

- detect-secrets (Yelp) — credential scanning across source code and config files
- garak (NVIDIA) — adversarial probing of LLM endpoints for injection and jailbreak susceptibility
- llm-guard (ProtectAI) — real-time scanning of LLM inputs and outputs for PII, injection, and toxicity
- checkov (Bridgecrew/Prisma Cloud) — IaC misconfiguration scanning with AI/ML service coverage

2. Getting Started

This section walks through installing DoofusAI SPM, starting the application, and running your first scan. No prior security experience is required.

2.1 System Requirements

Requirement	Details
Operating System	macOS 12+, Ubuntu 20.04+, or Windows 10/11 (WSL2 recommended on Windows)
Node.js	Version 18 or later (tested on Node 18, 20, 22, and 25)
Python (optional)	Python 3.9+ required only for open-source tool integrations (garak, llm-guard, detect-secrets, checkov)
Memory	Minimum 512 MB RAM; 1 GB recommended when running open-source tool wrappers
Disk space	200 MB for the application; additional space for the SQLite database (grows with scan history)
Network	Internet access not required; outbound calls only if scanning live external API endpoints

2.2 Installation

1. Download and unzip the package:

```
unzip doofusai-spm-v1.1.3.zip
cd doofusai-spm-v2
```

2. Install all dependencies. Run these commands in order from inside the project folder:

```
npm install                        # server and engine dependencies
cd ui && npm install && cd ..      # React dashboard dependencies
```

ⓘ **No C++ compiler or native build tools are required. DoofusAI SPM uses sql.js (pure WebAssembly SQLite) to avoid native compilation errors on newer versions of Node.**

3. Create your environment configuration file:

```
cp .env.example .env
```

Open the .env file in any text editor and fill in the relevant values. See Section 3 (Configuration) for a full description of every option.

4. Start the application:

```
npm run dev
```

You should see output like this within a few seconds:

```
[DB] Initialised — /your/path/data/doofusai.db
[API] DoofusAI SPM server running on http://localhost:3001
[API] OpenAPI docs: http://localhost:3001/api/docs
VITE v5.x  ready in 300ms
  Local:   http://localhost:5173/
```

5. Open http://localhost:5173 in your browser. The DoofusAI SPM dashboard will load.

⚠ **The DEP0169 deprecation warning about url.parse() is harmless. It comes from the swagger-ui-express library and does not affect functionality.**

2.3 Installing Optional Open-Source Tools

The four open-source tool integrations are optional. The application runs without them; checks simply report that the tool is not installed. To enable them:

```
pip install detect-secrets garak llm-guard checkov
```

ⓘ Each tool installs independently. You can install only the ones you need — for example, just detect-secrets for credential scanning without the full garak probing suite.

3. Configuration

DoofusAI SPM is configured through environment variables and YAML policy files. This section explains every option.

3.1 Environment Variables (.env)

The .env file in the project root controls how the application runs. Never commit this file to version control — it may contain API keys.

Server Settings

Variable	Default	Description
PORT	3001	Port the API server listens on. Change this if port 3001 is already in use on your machine.
DB_PATH	(auto)	Absolute path to the SQLite database file. Defaults to ./data/doofusai.db inside the project folder.
VITE_API_URL	http://localhost:3001	URL the browser dashboard uses to reach the API. Change this only if you move the API to a different host or port.

API Keys

API keys are never stored in the database. They are read from environment variables at runtime and injected into scans by name. Set the keys for whichever providers you plan to scan:

Variable	Provider
OPENAI_API_KEY	OpenAI (GPT-3.5, GPT-4, GPT-4o, etc.)
ANTHROPIC_API_KEY	Anthropic (Claude models)
AZURE_OPENAI_API_KEY	Azure OpenAI Service
AZURE_OPENAI_ENDPOINT	Azure OpenAI endpoint URL (e.g. https://my-resource.openai.azure.com/)
AWS_ACCESS_KEY_ID	AWS — used for SageMaker and S3 artefact checks
AWS_SECRET_ACCESS_KEY	AWS secret (always pair with AWS_ACCESS_KEY_ID)
AWS_REGION	AWS region (e.g. us-east-1)
GOOGLE_APPLICATION_CREDENTIALS	Path to a GCP service account JSON key file

⚠ **Never paste API keys directly into target configuration forms in the UI. Always set them here in the .env file and reference them by the variable name in your scan target config.**

3.2 Policy Files

Policies are YAML files that define what DoofusAI SPM checks and how it scores findings. Built-in policies live in engine/policy/. Your custom policies go in engine/custom-policies/ — this folder is excluded from version control by .gitignore.

Policy File Format

Every policy file must contain the following fields:

```
id: MY-CUSTOM-001                  # Unique identifier for this check
name: My custom security check     # Human-readable name
description: >                     # What this check detects
  Explain what the check looks for and why it matters.
severity: high                     # critical / high / medium / low / info
target_types: [llm_api]            # Which target types this applies to
check_module: checks/my-check      # Path to the JS check module (no .js)
enabled: true                      # Set false to disable without deleting
remediation: |                     # Step-by-step fix instructions
  1. First remediation step.
  2. Second remediation step.
framework_refs:                    # At least one framework mapping required
  - framework: OWASP_LLM
    id: LLM01
    url: https://owasp.org/...
  - framework: NIST_AI_RMF
    function: MEASURE
    category: MS-2.5
tags: [custom, my-team]            # Optional labels for filtering
```

Valid Values

Field	Allowed Values
severity	critical, high, medium, low, info
target_types	llm_api, ai_infra, model_registry, ai_app (can list multiple)
framework (in refs)	OWASP_LLM, MITRE_ATLAS, NIST_AI_RMF, CUSTOM

Field	Allowed Values
function (NIST)	GOVERN, MAP, MEASURE, MANAGE

Hot-Reloading Policies

You can add or edit policy files without restarting the server. After saving a new .yaml file to engine/custom-policies/, call the reload endpoint:

```
curl -X POST http://localhost:3001/api/v1/policies/reload
```

ⓘ **Custom policies with the same id as a built-in policy will override the built-in. This lets you customise remediation text or adjust severity without modifying the engine/policy/ directory.**

3.3 Writing a Custom Check Module

A check module is a JavaScript file placed in engine/checks/. It must export a run function that accepts a target and returns an array of findings:

```
// engine/checks/my-check.js
const { v4: uuidv4 } = require('uuid');

async function run(target, config) {
  const findings = [];

  if (!target.some_required_field) {
    findings.push({
      id: uuidv4(),
      title: 'Missing required field',
      description: 'Explain what was found and why it is a risk.',
      severity: 'high',       // critical/high/medium/low/info
      confidence: 'confirmed', // confirmed/probable/possible
      resource: target.name,
      evidence: 'some_required_field: null',
    });
  }

  return findings;
```

```
}

module.exports = { id: 'MY-CUSTOM-001', name: 'My check', run };
```

Each check runs in an isolated worker thread with a 30-second timeout. A check that crashes or hangs cannot affect other checks or the server.

4. Using the Dashboard

The DoofusAI SPM dashboard is a web application that runs at http://localhost:5173. It provides five main screens accessible from the top navigation bar.

4.1 Dashboard (Home Screen)

The home screen gives you an at-a-glance view of your overall AI security posture. It contains:

Posture Score Gauge

The circular gauge in the top-left shows your current posture score from 0 to 100. The score is calculated from all open findings using a weighted formula — each finding's severity and confidence level contribute to the penalty. The colour and tier label change based on the score:

Score Range	Tier	Meaning
90 – 100	Healthy	No significant open issues. Continue monitoring.
70 – 89	Needs Attention	Some issues present. Review and schedule remediation.
40 – 69	At Risk	Material exposure. Prioritise remediation of high and critical findings.
0 – 39	Critical	Severe exposure. Immediate action required on critical findings.

Summary Cards

Four metric cards appear at the top of the screen: Open Findings (total findings not yet resolved), Critical (open findings at critical severity), Scans (7d) (number of scans run in the past week), and Policies Active (number of enabled policies).

Charts

- Findings by Severity (bar chart) — shows the count of open findings at each severity level
- Findings by Framework (donut chart) — shows how findings are distributed across OWASP, MITRE ATLAS, and NIST AI RMF
- Findings by Target Type (stacked bar chart) — shows the breakdown of findings across LLM APIs, AI infrastructure, model registries, and AI apps
- Posture Score Trend (line chart) — shows how your score has changed over the past seven days

Top Critical Findings Table

The table at the bottom of the dashboard shows the five most severe open findings. Click any row to open the Finding Detail Drawer (see Section 4.3).

4.2 Scan Configuration Screen

Use this screen to configure and launch new scans. A three-step wizard guides you through the process.

Step 1 — Select Target Type

Choose the type of AI resource you want to scan from the four tiles: LLM / GenAI API, AI Infrastructure, Model Registry, or AI-Integrated App.

Step 2 — Configure Target

Fill in the target details. The fields shown depend on the target type selected:

Field	Description
Target name	A label for this target — used to identify it in scan results (e.g. prod-openai-gateway)
Host / endpoint URL	The URL or hostname of the resource being scanned
System prompt	(LLM API only) Paste the system prompt used by this endpoint. Leave blank to test for the absence of a system prompt.

⚠ **API keys must never be entered in the configuration form. Set them as environment variables in .env and the engine will read them automatically.**

Step 3 — Select Policies

Choose which security checks to run. Only policies applicable to the selected target type are shown. All enabled policies are selected by default. You can deselect individual policies to skip specific checks.

Click Run Scan Now to start the scan. The scan runs in the background; the Recent Scans table at the top of the screen updates automatically when it completes.

4.3 Findings Screen

The Findings screen lists all security findings across all scans. You can filter, sort, and act on findings from this screen.

Filtering Findings

Use the filter bar at the top of the screen to narrow the list:

- Search box — type any part of a finding title or resource name
- Severity filter — click a severity level button to show only findings at that level
- Status filter — show only open, acknowledged, or resolved findings
- Target filter — show only findings for a specific target type

Sorting

Click any column header to sort by that column. Click again to reverse the sort order. A small arrow indicates the active sort column and direction.

Finding Detail Drawer

Click any row to open the detail drawer on the right side of the screen. The drawer shows:

- Full description of the finding and what it means
- Affected resource (automatically redacted to remove any sensitive values)
- Evidence — the specific configuration value or probe result that triggered the check
- Framework Mapping — clickable badges that link directly to the relevant OWASP, MITRE ATLAS, or NIST documentation
- Remediation Steps — numbered, copy-pasteable instructions for fixing the issue

Acting on Findings

Three action buttons appear at the bottom of the detail drawer:

Button	Effect
Mark Resolved	Records the finding as fixed. Resolved findings are excluded from the posture score calculation.
Acknowledge	Records that you are aware of the finding but have not yet fixed it. Acknowledged findings still count toward the posture score.
Reopen	Returns a resolved or acknowledged finding to open status.

4.4 Policy Editor Screen

The Policy Editor provides a built-in YAML editor for creating and validating custom policies directly in the browser. You do not need to edit files manually.

- Edit the YAML in the left panel. The editor pre-fills a template to guide you.
- Click Validate to check your policy against the schema. Errors are shown below the editor with specific field names.

- Once valid, save the file to engine/custom-policies/ and click the reload button or call the reload API endpoint.

The right panel shows a Schema Reference (all required and optional fields with their types) and a list of currently active policies with their enabled/disabled status.

4.5 Reports Screen

The Reports screen generates a formatted security report from any completed scan.

Generating a Report

1. Select a completed scan from the Scan dropdown
2. Optionally filter by severity to include only findings above a certain level
3. Review the report preview, which shows each finding with its description and first remediation step

Exporting

Export Option	Description
Download JSON	Exports the full report as a structured JSON file including all finding details, framework references, and remediation steps. Suitable for integration with other tools.
Copy as Text	Copies the entire report to your clipboard as plain text. Suitable for pasting into email, tickets, or chat.

5. Built-In Security Checks

DoofusAI SPM includes 12 built-in checks and 4 open-source tool wrappers. This section describes each one.

Check ID	Target	Framework	What It Detects
OWASP-LLM01	LLM API / App	OWASP LLM01	Missing or unprotected system prompt; direct prompt injection vulnerability
OWASP-LLM06	LLM API	OWASP LLM06 / NIST	Exposed API keys in config; missing rate-limit guardrails
INFRA-001	AI Infrastructure	OWASP LLM05 / ATLAS	Unauthenticated MLflow or Jupyter endpoints reachable without credentials
INFRA-002	AI Infrastructure	NIST RMF / ATLAS	Unencrypted model artefact storage; publicly readable S3/GCS buckets; no network isolation
REGISTRY-001	Model Registry	OWASP LLM05 / ATLAS	Unsigned model artefacts; pickle-format models vulnerable to code execution on load
REGISTRY-002	Model Registry	OWASP LLM05 / NIST	Missing SBOM; no licence on public model cards; undocumented training data provenance
APP-001	AI App	OWASP LLM08 / ATLAS	Excessive agent tool access; no human-in-the-loop; no recursion depth limit
APP-002	AI App	OWASP LLM06 / NIST	Insecure RAG retrieval; no PII filtering; unsanitised inputs injected to vector DB
TOOL-DETECT-SECRETS	All targets	OWASP LLM06 / NIST	Credentials, tokens, and secrets found in source code or configuration files
TOOL-GARAK	LLM API / App	OWASP LLM01 / ATLAS	LLM endpoint fails garak probes: prompt injection, jailbreaks, data extraction
TOOL-LLM-GUARD	LLM API / App	OWASP LLM01, LLM06	Unsafe content in prompts or outputs: PII, injection, toxicity, banned topics
TOOL-CHECKOV	AI Infrastructure	OWASP LLM05 / NIST	IaC misconfigurations in Terraform, CloudFormation, or Kubernetes for ML services

5.1 Severity and Scoring

Each finding is assigned a severity level and a confidence level. Together they produce a CVSS-style composite score from 0 to 10:

Severity	Score	Meaning	Typical Action
CRITICAL	10.0	Actively exploitable, immediate risk	Fix within 24 hours
HIGH	8.0	Significant exposure, likely exploitable	Fix within 7 days
MEDIUM	5.0	Elevated risk requiring attention	Fix within 30 days
LOW	2.0	Minor issue, limited impact	Fix in next sprint
INFO	0.0	Informational — no immediate risk	Log and monitor

Confidence levels modify the severity score: Confirmed multiplies by 1.0, Probable by 0.75, and Possible by 0.5. For example, a high-severity finding with probable confidence scores 8.0 × 0.75 = 6.0.

The overall posture score is calculated as:

```
Posture Score = max(0,  100 - (sum of all open finding scores × 1.5))
```

ⓘ **Resolved findings are excluded from the posture score. Acknowledged findings are included — acknowledging a finding records awareness but does not credit the remediation.**

6. REST API Reference

All dashboard functionality is available via REST API. The interactive API documentation (Swagger UI) is available at http://localhost:3001/api/docs while the server is running.

6.1 Endpoints

Method	Path	Description
GET	/api/v1/health	Health check — confirms the server is running and returns the version number
POST	/api/v1/scans	Start a new scan. Returns a scan_id immediately; poll GET /scans/:id for results.
GET	/api/v1/scans	List recent scans. Optional query parameter: limit (default 20, max 100).
GET	/api/v1/scans/:id	Get scan status and all findings for a specific scan.
GET	/api/v1/findings	List findings. Filter by scan_id, severity, status, target_type, or limit.
PATCH	/api/v1/findings/:id/status	Update a finding status to open, acknowledged, or resolved.
GET	/api/v1/reports/:scan_id	Get a posture report. Add ?format=summary for a condensed view.
GET	/api/v1/policies	List all currently loaded policies.
POST	/api/v1/policies/reload	Hot-reload policies from disk without restarting the server.

6.2 Starting a Scan via API

Send a POST request to /api/v1/scans with a target object:

```
curl -X POST http://localhost:3001/api/v1/scans \
  -H 'Content-Type: application/json' \
  -d '{
    "target": {
      "type": "llm_api",
      "name": "my-openai-endpoint",
      "model_name": "gpt-4o",
      "api_key_env": "OPENAI_API_KEY"
    }
  }'
```

The response includes a scan_id. Poll GET /api/v1/scans/:id until the status field changes from running to completed.

6.3 Target Configuration Fields

LLM API Target

Field	Required	Description
type	Yes	Must be llm_api
name	Yes	Human-readable label for this target
model_name	No	Model identifier (e.g. gpt-4o). Required for garak probing.
api_key_env	No	Name of the environment variable holding the API key (e.g. OPENAI_API_KEY)
system_prompt	No	The system prompt text. Leave blank to test for absence.
rate_limit_rpm	No	Configured requests-per-minute limit. Absence triggers a finding.
sample_prompts	No	Array of prompt strings for llm-guard scanning

AI Infrastructure Target

Field	Required	Description
type	Yes	Must be ai_infra
name	Yes	Human-readable label
host	No	Hostname or IP to probe for unauthenticated endpoints
artifact_bucket	No	S3 or GCS bucket name for artefact storage checks
sse_enabled	No	true/false — whether server-side encryption is enabled on the bucket
bucket_public	No	true/false — whether the bucket has a public ACL
audit_log_enabled	No	true/false — whether audit logging is configured
vpc_isolated	No	true/false — whether the infrastructure is in a private VPC
iac_path	No	Path to IaC directory for checkov scanning

Model Registry Target

Field	Required	Description
type	Yes	Must be model_registry

Field	Required	Description
name	Yes	Human-readable label
artefact_formats	No	Array of file extensions in use (e.g. ["pkl", "pt", "safetensors"])
signature_verification	No	true/false — whether model signatures are verified on load
versioning_policy	No	true/false — whether a versioning policy is enforced
public	No	true/false — whether the registry is publicly accessible
sbom_present	No	true/false — whether an SBOM exists for models in this registry
licence	No	SPDX licence identifier (e.g. Apache-2.0). Absence on public models triggers a finding.

AI App Target

Field	Required	Description
type	Yes	Must be ai_app
name	Yes	Human-readable label
tools	No	Array of tool names available to the agent (e.g. ["send_email", "read_file"])
human_in_the_loop	No	true/false — whether high-risk actions require human approval
max_recursion_depth	No	Maximum number of agent iterations (absence or >20 triggers a finding)
vector_db	No	true/false — whether a vector database is used for retrieval
vector_db_auth	No	true/false — whether the vector DB requires authentication
pii_filter_enabled	No	true/false — whether PII is scanned before or after LLM calls
input_validation	No	true/false — whether user inputs are validated before reaching the LLM

7. Troubleshooting

7.1 Common Errors and Fixes

Error / Symptom	Solution
sh: concurrently: command not found	Run npm install from the project root folder before npm run dev.
Cannot find module 'uuid' (or js-yaml, zod, etc.)	Run npm install from the project root (not from inside api/ or ui/).
TypeError: initSqlJs.Database is not a constructor	You have an older version of the db.js file. Download the latest v1.1.3 package.
better-sqlite3 compilation error with node-gyp	You have an older version of the package. v1.1.1+ uses sql.js and requires no compilation.
Port 3001 already in use	Set PORT=3002 (or any free port) in your .env file, then restart.
Dashboard shows no data / blank charts	Check that VITE_API_URL in .env matches the port the API is running on. Default is http://localhost:3001.
DEP0169 deprecation warning in console	Harmless. This comes from swagger-ui-express and does not affect functionality.
Open-source tool check shows 'not installed'	Install the tool with pip install <tool-name>. See Section 2.3 for the full install command.
Scan shows 0 findings	Check that the target type in your scan config matches the policies selected. Verify the check modules exist in engine/checks/.

7.2 Checking Server Logs

All server activity is logged to the terminal where you ran npm run dev. The [0] prefix indicates API server output; [1] indicates the Vite UI server. Look for lines prefixed with [DB], [API], [Scanner], [Runner], or [PolicyLoader] for diagnostic information.

To start only the API server (for debugging without the UI):

```
node api/server.js
```

7.3 Resetting the Database

The SQLite database is stored at data/doofusai.db. To reset all scan history and findings, stop the server and delete this file:

```
rm data/doofusai.db
npm run dev    # the database will be recreated automatically on next start
```

⚠ **Deleting the database permanently removes all scan history and finding statuses. This cannot be undone.**

8. Security and Privacy

8.1 How Secrets Are Protected

DoofusAI SPM is designed so that sensitive credentials cannot leak through the tool itself:

- API keys are read exclusively from environment variables. They are never accepted through the UI form, never stored in the database, and never appear in log output.
- All evidence strings and resource identifiers are automatically redacted before being written to the database. The redaction function removes API key patterns, Bearer tokens, email addresses, and phone numbers.
- Each check runs in an isolated worker thread. A check module cannot access environment variables or other checks' data.

8.2 Data That Stays Local

DoofusAI SPM does not make any outbound network calls except to the targets you explicitly configure for scanning. No telemetry, no analytics, no licence checks. All scan data, findings, and reports remain in the local SQLite database on your machine.

8.3 Running in Production

For production deployments (e.g. running DoofusAI SPM as a shared team service), consider the following additional steps:

- Place the API server behind a reverse proxy (nginx, Caddy) with HTTPS and authentication
- Set DB_PATH to a persistent volume path outside the application directory
- Use a secrets manager (AWS Secrets Manager, HashiCorp Vault) to inject environment variables rather than a .env file
- Run the application as a non-root system user
- Enable access logging on the reverse proxy to maintain an audit trail of who ran which scans

9. Glossary

Term	Definition
AI Agent	An LLM-based system that can take autonomous actions using tools such as web search, code execution, or API calls.
ATLAS (MITRE)	Adversarial Threat Landscape for Artificial-Intelligence Systems — a knowledge base of adversarial ML attack techniques published by MITRE.
CVSS	Common Vulnerability Scoring System — a standardised method for rating the severity of security vulnerabilities, scored 0 to 10.
garak	An open-source LLM vulnerability scanner by NVIDIA that probes model endpoints for susceptibility to prompt injection, jailbreaks, and other attacks.
IaC	Infrastructure-as-Code — defining cloud infrastructure (servers, storage, networking) in configuration files such as Terraform or CloudFormation.
LLM	Large Language Model — a deep-learning model trained on text data capable of generating, summarising, translating, and reasoning about text (e.g. GPT-4, Claude).
MLOps	Machine Learning Operations — practices and tools for deploying, monitoring, and managing ML models in production (e.g. MLflow, Kubeflow, SageMaker).
NIST AI RMF	NIST Artificial Intelligence Risk Management Framework — a voluntary framework for managing risk in AI systems, organised into four functions: Govern, Map, Measure, Manage.
OWASP LLM Top 10	A list of the ten most critical security risks for LLM-based applications, published by the Open Web Application Security Project.
Pickle	A Python serialisation format commonly used to store ML model weights. Loading a pickle file executes arbitrary Python code, making it a significant supply-chain risk.
Posture Score	A 0–100 score representing the overall security health of the scanned AI infrastructure. Higher is better. Calculated from the severity and confidence of open findings.
Prompt Injection	An attack where an adversary embeds malicious instructions in data processed by an LLM (user input, retrieved documents, web pages) to override the model's intended behaviour.
RAG	Retrieval-Augmented Generation — a pattern where an LLM's response is grounded by retrieving relevant documents from a database or search engine before generating a response.
SBOM	Software Bill of Materials — a formal inventory of all components, libraries, and dependencies in a software artefact, including ML models.
SPM	Security Posture Management — the practice of continuously assessing, monitoring, and improving the security configuration of a set of infrastructure or services.

Term	Definition
Vector Database	A database optimised for storing and searching embedding vectors, commonly used in RAG pipelines to retrieve semantically relevant documents.

10. About

10.1 Author

DoofusAI SPM was designed, architected, and built by Shahryar Jahangir. The product reflects Shahryar's work across cloud security, AI infrastructure, and developer tooling, and is released as a production-grade open platform for security and engineering teams to build upon.

10.2 Version History

Version	Date	Changes
1.1.3	March 2026	Fixed sql.js async initialisation; fixed module resolution for engine dependencies at root; corrected concurrently install path
1.1.2	March 2026	Consolidated all server and engine dependencies into root package.json to resolve MODULE_NOT_FOUND errors
1.1.1	March 2026	Moved concurrently to runtime dependencies; removed workspace config; added npm run install:all shortcut
1.1.0	March 2026	Replaced better-sqlite3 with sql.js (pure WebAssembly) to eliminate C++ compilation failures on Node 25 / macOS
1.0.0	March 2026	Initial release: engine core, 8 built-in checks, 4 open-source tool wrappers, REST API, full React dashboard

10.3 Licence

DoofusAI SPM is released for internal use. The open-source tools integrated as check modules (garak, llm-guard, detect-secrets, checkov) are governed by their respective licences.

This document and all associated source code are the intellectual property of Shahryar Jahangir.